

Alexandre KLEIN

NFE204

Projet CNAM NFE204

**Projet : Déterminer par les techniques associées aux Big Datas
les éléments suscitant le bonheur chez les Européens :**

<http://happy.inmyworlds.com/index.html>

Plan :

Contents

Introduction :	3
Description du projet :	3
Choix technologiques	3
L'API Twitter :	3
Google Compute Engine, Linux, Apache, PHP, MongoDB, Google MAP Api, jquery.	4
Google Cloud Engine	4
Linux, Apache 2.2, PHP, librairies php, MapQuest API, Google MAP API, jquery	4
MongoDB 2.4	4
Elasticsearch	4
La percolation :	5
Les agrégations :	5
Implémentation	6
Architecturale :	6
Description :	6
Schéma :	6
Exemples de configurations :	7
Architecture Logicielle:	10
Description :	10
La mise en place des index elasticsearch :	10
Le script de récupération des données :	14
Le webservice d'envoi des données agrégées :	18
Le webservice d'abonnement :	20
La page Web :	22
Conclusion	24

Introduction :

Pas un jour ne passe sans qu'un article de presse ne parle de Big Data. Comment le Big Data a changé les attitudes et stratégies de nos entreprises, comment le Big Data fait avancer la médecine, comment le Big data est au cœur du marketing, de la politique, de l'avenir... et du présent.

Le « Big Data », terme en vogue s'il en est, regroupe un ensemble de concepts concernant le stockage et le traitement analytique d'un nombre très important de données. Une telle masse de donnée ne permet plus aux outils traditionnels de la traiter dans un temps acceptable : l'heure est à la distribution des stockages, distribution des calculs et aux PAAS/SAAS/IAAS basés sur ces deux concepts, socles de bases rendant aujourd'hui le Big Data accessibles aux petites ou grandes entreprises, aux étudiants.

Aussi est-il aujourd'hui possible de stocker des masses gigantesques de données pour leur trouver une utilité que ce soit en temps réel, ou a posteriori.

Si cette technologie est aujourd'hui accessible, pourquoi ne l'utiliser pour rechercher ce qui rend les gens heureux ? Faire une analyse géographique de ce qui met les européens en joie à un moment donné, en un lieu donné, de l'afficher en temps réel et de permettre une analyse a posteriori des raisons intrinsèques de ce bonheur ?

Description du projet :

Twitter est une source de données incroyablement variée, et complexe. S'il est possible d'utiliser leurs API afin de pouvoir trouver facilement des tweets par recherche, il est difficile de les traiter et de les analyser directement. Ces données sont de plus polluées par des spammeurs qui utilisent certains mots clefs pour faire apparaître leurs liens ou publicités au près du plus grand nombre.

Afin de pouvoir effectuer des traitements analytiques nous allons devoir stocker des informations qualifiées venant de twitter dans une base de données, les indexer dans un moteur de recherche et ensuite agréger ces données pour visualiser des informations pertinentes.

Dans le cas présent nous allons chercher à extraire de twitter des twitts liés au bonheur en anglais et en français, essayer de filtrer les spams, les stocker avec les coordonnées géographiques de l'utilisateur de twitter ayant laissé un message pour ensuite agréger ces données par localisation géographique pour afficher ce qui semble générer du bonheur dans telle ou telle partie du monde.

Choix technologiques

L'API Twitter :

Twitter enregistre chaque jour une masse incroyable de données issues des pensées et des envies de communications de plusieurs millions d'utilisateurs qui exposent sur ce medium leur réflexions, leurs états d'âmes, leur réactions aux sujets d'actualité, à ce qui les rends joyeux, en temps réel. Ces messages peuvent être requêtés, stockés et analysés en utilisant des API mises à disposition par l'entreprise

L'API search proposée par twitter permet de se connecter à cette gigantesque masse d'information avec les limitations suivantes : 180 Requêtes par fenêtre de 15 minutes. Cette limite ne sera pas bloquante dans le cadre des besoins de ce projet.

[Google Compute Engine, Linux, Apache, PHP, MongoDB, Google MAP Api, jquery.](#)

[Google Cloud Engine](#)

Mettre en place un service Big Data suppose une infrastructure adaptée sur laquelle pouvoir stocker les données et paralléliser les traitements. Google Compute Engine est une infrastructure à la demande (IAAS) adaptée à ce genre de besoin. De plus, la version d'essai proposant 300€ de crédit pour tester la solution nous serons en mesure de créer la plateforme et faire les tests et développements idoines.

Accessible via API comme par une interface web, elle va me permettre de mettre en place facilement des loadbalancers pour les elasticsearch, frontaux et mongos.

[Linux, Apache 2.2, PHP, librairies php, MapQuest API, Google MAP API, jquery](#)

Linux est un système d'exploitation libre et gratuit largement utilisé dans le cadre de projets Big Data. Nativement prise en charge par Google GCE, Debian Wheezy sera donc la distribution de base pour les instances créées dans le cadre de ce projet.

Pour des questions de confort et d'habitude, le serveur Web et le langage de programmation seront respectivement Apache 2.2 et PHP 5.4.

Afin de faciliter le développement de l'application nous allons utiliser le Wrapper twitter-api-php pour faciliter l'utilisation de l'API Search twitter, nous allons requêter sur l'api MapQuest afin de traduire des lieux en coordonnées géographiques, et nous allons aussi avoir besoin d'une librairie geohash afin de décoder ceux renvoyés par elasticsearch lors des requêtes par géodistance.

L'API google MAP sera utilisée pour générer la carte, tandis que jquery sera mis à profit pour l'ajax.

[MongoDB 2.4](#)

Afin de stocker les données récupérées via l'API twitter nous allons avoir besoin d'un moteur de base de données hautement scalable. Le choix s'est porté sur mongoDB pour ses capacités de sharding et ses facilités d'indexation. De plus l'existence d'une river mongoDB pour elasticsearch facilitera les développements ultérieurs.

[Elasticsearch](#)

Elasticsearch est un moteur de recherche libre basé sur l'index Lucène créé pour pouvoir être très facilement distribué. Il dispose de très nombreuses fonctionnalités pertinentes pour le projet telles que :

La percolation :

Quand on pense à un moteur de recherche nous pensons généralement à la recherche au sein de documents stockés que l'on requêtera selon les informations que l'on estime nécessaires. Les données sont donc stockées dans une base, organisée de façon à pouvoir en indexer les documents pertinents, éliminer le superflu, dans l'attente d'être potentiellement utilisées par la suite.

Elasticsearch permet de faire exactement l'inverse : La percolation.

Au sein d'Elasticsearch, tout est document. Des données stockées aux requêtes permettant d'extraire les données : tout est Json. La percolation consiste donc à stocker non pas les informations mais les requêtes elle mêmes.

Cette capacité permet d'utiliser les données beaucoup plus facilement dans le cadre de programmation événementielle ou lorsque l'on a besoin de prendre des décisions.

Dans le cadre de ce projet je vais avoir besoin de supprimer les documents qui se révèlent être du spam. Je vais donc devoir analyser mes tweets afin de déterminer s'ils contiennent des éléments ou une url qui m'indiquerait qu'il n'est pas nécessaire de les stocker.

L'ensemble des mots clefs ou des url qui ne doivent pas être stockées en base sera donc placée au sein d'Elasticsearch sous la forme de requêtes et, à la réception de données je vais les « percoler » contre mon index de requêtes et vérifier si au moins une des requêtes s'avère correspondre. Si tel est le cas la donnée ne sera pas insérée.

Si la percolation est utile dans le cadre du décisionnel, elle peut aussi l'être dans le cadre de l'événementiel. Ainsi sera-t-il possible de s'abonner à un type de document, et être notifié à chaque fois qu'un événement heureux est notifié dans sa ville.

Les agrégations :

La création des agrégations au sein d'Elasticsearch est née des limitations des facets : Il est impossible de faire des facets de facets.

Les agrégations permettent de le faire. Ainsi, dans ce projet, nous allons devoir agréger les données des mots les plus associés au bonheur pour une zone géographique donnée.

Implémentation

La mise en place d'une solution applicative, NoSQL ou non, suppose une réflexion approfondie sur la façon dont nous allons répondre aux différents requêtes adressées au service tout en maximisant les ressources dont nous disposons. Elle est basée sur deux environnements intrinsèquement mêlés : La vision architecturale « hardware » et la vision architecturale du point de vue logiciel.

Architecturale :

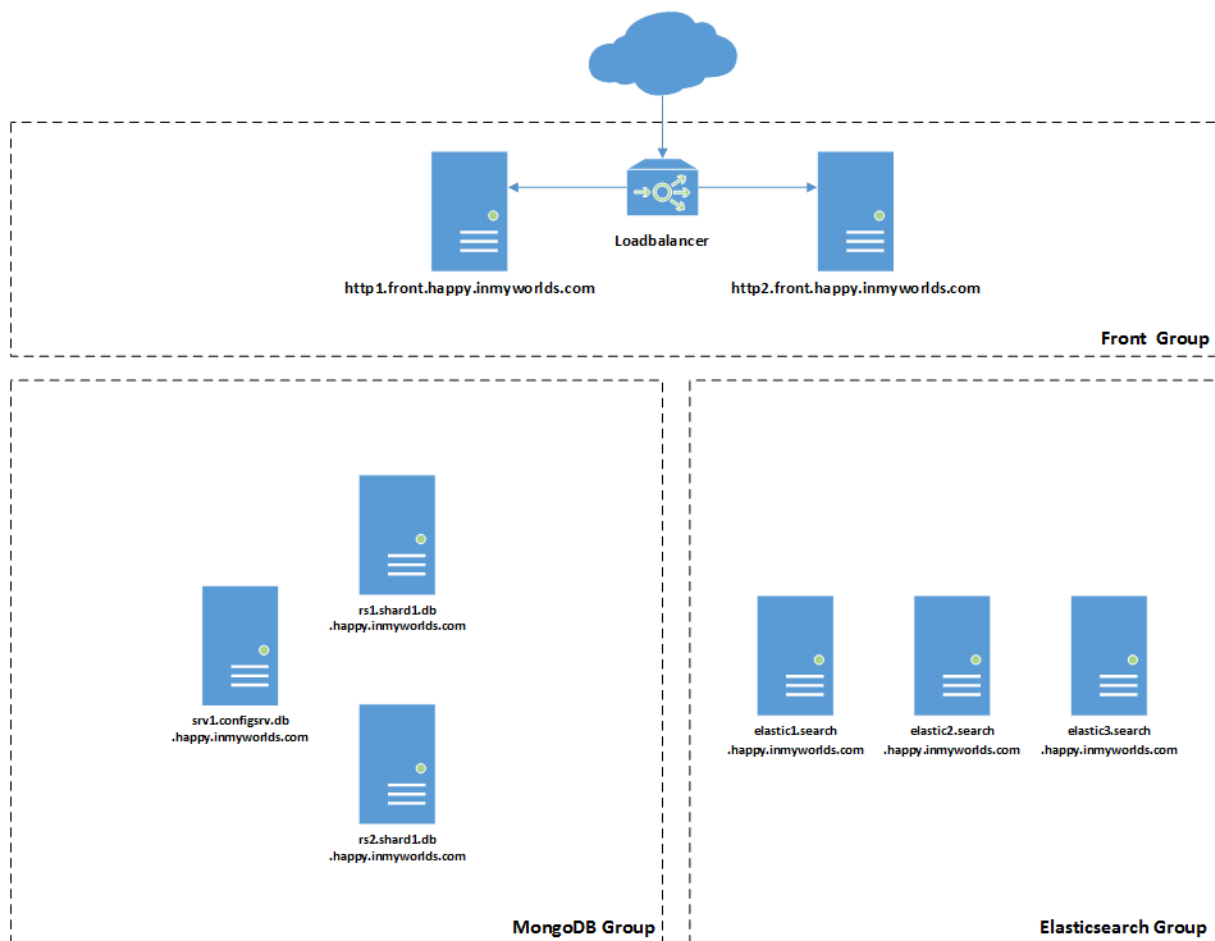
Description :

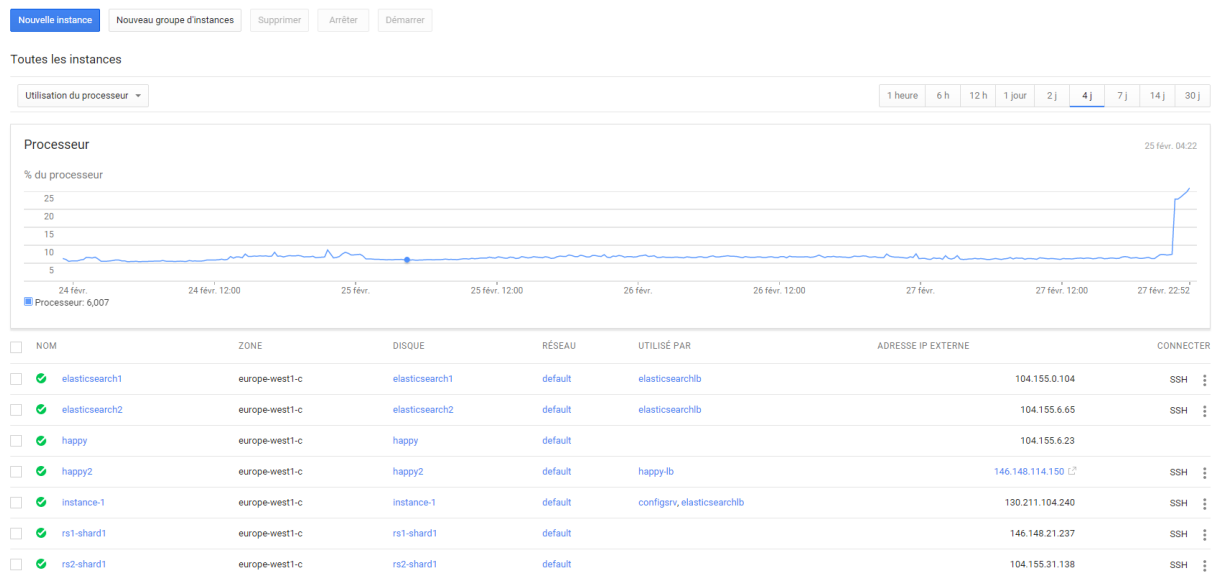
Nous avons décidé de nous baser sur une architecture résiliente permettant de lisser les entrées en écriture et de scaler les lectures. Google Cloud Engine a servi de support à cette architecture.

Notre architecture cible sera donc constituée de

- Deux serveurs fronts contenant deux fronts apache et deux mongos.
- Trois serveurs elasticsearch.
- 4 Serveurs MongoDB dont :
 - 1 Config Servers Contenant un mongoDB Arbiter.
 - Un réplicaset de deux machines.

Schéma :





Afin de pouvoir installer une river entre mongoDB et elasticsearch le plugin elasticsearch-river-mongodb a été installé sur chacun des 3 serveurs elasticsearch

Afin de préparer l'avenir une scalabilité horizontale sera nécessaire pour s'adapter au grossissement des requêtes sur les elasticsearch et sur les frontaux.

Pour cela Deux LoadBalancers ont été mis en place :

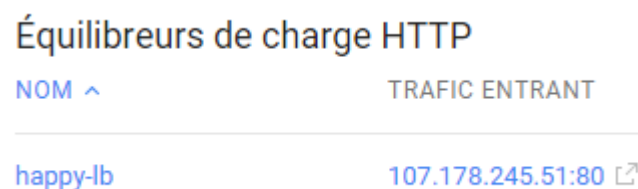
- Un premier pour les Elasticsearch.

CONFIGURATION DE BASE [RÈGLES DE TRANSFERT](#) POOLS CIBLES TESTS PÉRIODIQUES

[Nouvelle règle de transfert](#)
[Supprimer](#)

NOM	RÉGION	ADRESSE IP	PROTOCOLE/PORT
☐ lb1-rule	europa-west1	104.155.20.90	TCP/9200

- Un second pour les fronts Web.



Ces adresses seront déclarées sur les fichiers hosts afin de permettre l'équilibrage de charge.

La routine de récupération des données sera quant à elle effectuée sur le serveur http1.

Exemples de configurations :

Mongos.conf du serveur http1.front.happy.inmyworlds.com :

```
configdb=srv1.configsrv.db.happy.inmyworlds.com:27019, srv2.configsrv.db.happy.inmyworlds.com:27019,
srv3.configsrv.db.happy.inmyworlds.com:27019

logpath=/srv/log/mongo/mongos/mongos.log

logappend=true

fork=true

port=27017

quiet=true
```

Mongo-database.conf du serveur rs1.shard1.db.happy.inmyworlds.com :

```
logpath=/srv/log/mongo/db/mongod.log

logappend=true

port = 27017

fork = true

dbpath=/srv/mongo/db/

nprealloc = true

replSet = rs1

shardsvr = true

quiet=true

logpath=/srv/log/mongo/db/mongod.log

logappend=true

port = 27017

fork = true

dbpath=/srv/mongo/db/

nprealloc = true

replSet = rs1

shardsvr = true

quiet=true
```

Mongo-configsrv.conf du serveur srv1.configsrv.happy.inmyworlds.com :

```
logpath=/srv/log/mongo/config-server/config-server.log

logappend=true

port = 27019

fork = true

dbpath=/srv/mongo/config-server

nprealloc = true

configsvr=true

quiet=true
```


Mongo-arbiter.conf du serveur srv1.configsrv.happy.inmyworlds.com :

```
logpath=/srv/log/mongo/mongo-arbiter/arbiter.log

logappend=true
cluster.name: happycluster

node.name: "elastic1"

discovery.zen.ping.unicast.hosts: ["elastic1.search.happy.inmyworlds.com:9300", "elastic2.search.happy.inmyworlds.com:9300",
"elastic3.search.happy.inmyworlds.com:9300"]

.

replSet = rs1

cluster.name: happycluster

node.name: "elastic2"

discovery.zen.ping.unicast.hosts: ["elastic1.search.happy.inmyworlds.com:9300", "elastic2.search.happy.inmyworlds.com:9300",
"elastic3.search.happy.inmyworlds.com:9300"]
```

Elasticsearch.yaml du serveur elastic3.search.happy.inmyworlds.com :

```
cluster.name: happycluster

node.name: "elastic3"

discovery.zen.ping.unicast.hosts: ["elastic1.search.happy.inmyworlds.com:9300", "elastic2.search.happy.inmyworlds.com:9300",
"elastic3.search.happy.inmyworlds.com:9300"]
```

Fichier hosts du serveur elastic1.search.happy.inmyworlds.com :

```
127.0.0.1 localhost

::1 localhost ip6-localhost ip6-loopback

fe00::0 ip6-localnet

ff00::0 ip6-mcastprefix

ff02::1 ip6-allnodes

ff02::2 ip6-allrouters

10.240.84.95 rs1.shard1.db.happy.inmyworlds.com

10.240.94.173 rs2.shard1.db.happy.inmyworlds.com

10.240.108.185 arb.shard1.db.happy.inmyworlds.com

10.240.137.173 elasticsearch1.c.firm-link-858.internal elasticsearch1 # Added by Google

10.240.137.173 elastic1.search.happy.inmyworlds.com

10.240.79.186 elastic2.search.happy.inmyworlds.com

10.240.108.185 elastic3.search.happy.inmyworlds.com
```

Architecture Logicielle:

Description :

L'application est découpée en deux composants :

- Le premier sert à aller chercher les données pertinentes sur twitter et à les injecter dans une base mongoDB. Ces données seront remontées au sein d'elasticsearch par le biais d'une river.
- Le second est le site internet. Permettant lors de la saisie d'une date de récupérer les informations et de les afficher

La mise en place des index elasticsearch :

L'index twitter :

La page pour afficher Les twitts nécessite composants : Un Web Service pour renvoyer le json pour la date voulue.

Dans un premier nous allons devoir déclarer l'index au sein d'elasticsearch pour pouvoir indexer les tweets :

Cette déclaration de l'index se décompose en deux partie :

- Les mappings qui décrivent les champs, leur format et l'analyseur à utiliser
- Les settings qui décrivent les analyseurs et filtres à utiliser.

Je vais donc indexer au sein d'elasticsearch : l'id du tweet, le texte du tweet, sa date de création au format utilisé par twitter, sa localisation textuelle et ses coordonnées géographiques. Ces dernières sont déclarées en tant que geo_point me permettant ainsi de faire une agrégation dessus par la suite.

A cause des limitations de l'API twitter gratuite, nous allons nous concentrer sur les tweets anglais et français.

Je choisis le tokenizer standard auquel j'ajoute différents filtres tels qu'une liste de stopwords contenant les mots qui m'ont permis de récupérer les tweets et qui ne sont donc plus représentatifs des informations stockées ainsi que les mots usuels de twitter sans intérêts dans le cas présent.

Il faut aussi supprimer les mots inutiles sémantiquement en anglais et en français, les accents, et les mettre en minuscule, supprimer les élisions. Je n'aurai ainsi aucun doublon quand viendra le moment d'agréger les mots les plus couramment utilisés.

L'ensemble de ces filtres ne doivent s'appliquer qu'au champ contenant le tweet à proprement parler.

Le json envoyé au serveur elasticsearch est donc le suivant :

```

curl -s -XPUT "http://localhost:9200/twitter" -d '
{
  "settings": {
    "analysis": {
      "analyzer": {
        "twittsTokenizor" : {
          "type" : "custom",
          "tokenizer" : "standard",
          "filter" : [
            "french_elision",
            "asciifolding",
            "lowercase",
            "french_stop",
            "english_stop",
            "twitter_stop"
          ]
        }
      },
      "filter" : {
        "french_elision" : {
          "type" : "elision",
          "articles" : [ "l", "m", "t", "qu", "n", "s",
            "j", "d", "c", "jusqu", "quoiqu",
            "lorsqu", "puisque"
          ]
        },
        "asciifolding" : {
          "type" : "asciifolding"
        },
        "french_stop": {
          "type": "stop",
          "stopwords": "_french_"
        },
        "english_stop": {
          "type": "stop",
          "stopwords": "_english_"
        },
        "twitter_stop" : {
          "type" : "stop",
          "stopwords": ["RT", "FAV", "TT", "FF", "http", "rt",
            "content", "Heureux", "heureuse", "joie",
            "bonheur", "youpi", "content", "happy", "cheerful",
            "delighted", "ecstatic", "joyful", "pleased", "glad",
            "t.co", "http", "https"]
        }
      }
    },
    "mappings": {
      "twitter": {
        "properties": {
          "id": {
            "type": "long",
            "store": true
          },
          "text": {
            "type": "string",
            "store": true,
            "analyzer" : "twittsTokenizor"
          },
          "created_at": {

```

```

        "type": "date",
        "format": "EE MMM d HH:mm:ss Z yyyy",
        "store": true
    },
    "location": {
        "type": "string",
        "store": true
    },
    "geo": {
        "type": "geo_point",
        "store": true
    }
}
}
}
}

```

Viens ensuite la déclaration de ma river :

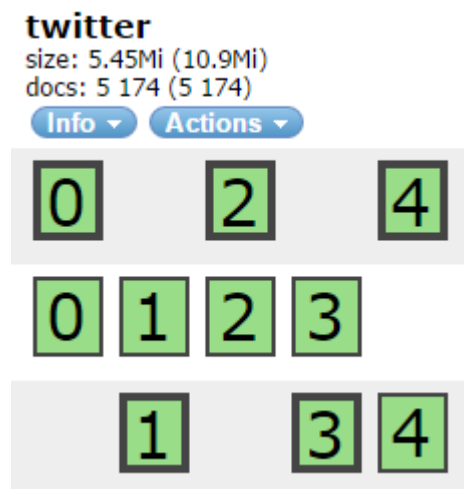
```

{
  "type": "mongodb",
  "mongodb": {
    "servers": [{
      "host": "mongos.db.happy.inmyworlds.com", "port": 27017
    }],
    "options": {},
    "db": "twitter",
    "collection": "tweets"
  }, "index": {
    "name": "twitter"
  }
}

```

J'y défini l'emplacement de mon mongos, son port, la base de données mongo, le nom de la collection et l'index twitter dans laquelle la river va se déverser.

L'index vu par le plugin head :



Le percolateur de Blacklists :

En regardant les différents tweets se déversant dans l'élasticsearch il est apparu que de nombreux spams étaient intégrés, contenant des liens vers des sites frauduleux ou adultes. Il a donc été nécessaire de définir un moteur de décision pour intégrer ou non les tweets dans la base de données.

Pour cela l'utilisation d'un percolateur a du sens : De nombreuses blacklists sont disponibles sur internet. Pour cela nous allons récupérer celle fournie par shallalist.de et l'insérer dans un nouvel index appelé blacklist.

Le Mapping correspond au besoin de vérifier un champ ou les adresses emails et urls seront préservées : Les autres tokenizer par défaut retirent les points.

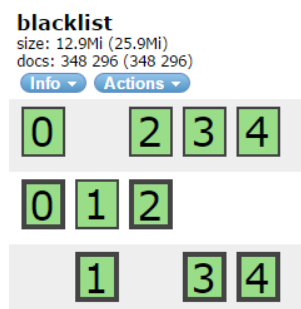
Le Mapping:

```
curl -XPUT 'localhost:9200/blacklist' -d '{
  "domains": {
    "properties": {
      "domain": {
        "type": "string",
        "analyzer" : "notoken"
      }
    }, "analysis": {
      "analyzer": {
        "notoken" : {
          "type" : "uax_url_email"
        }
      }
    }
  }
}
```

L'intégration des query au sein du percolateur se fera par une commande bash :

```
wget http://www.shallalist.de/Downloads/shallalist.tar.gz
tar -xvzf shallalist.tar.gz
cd BL; rm [...]
find . -type f -name "domains" -exec cat {} \; > /tmp/list
i=1; while read line; do curl -XPUT
elastic.search.happy.inmyworlds.com:9200/blacklist/.percolator/$i -d "{
\"query\" : { \"match\" : {\"domain\" : \"$line\" } } }"; (( i = $i + 1 ));
sleep 0.001 ; done < /tmp/list
```

L'index de requêtes une fois rempli, vu par le plugin head :



Le percolateur d'abonnements :

Afin de pouvoir notifier aux utilisateurs les tweets de leur zone nous allons devoir mettre en place la aussi une mécanique décisionnelle : Comment savoir si un utilisateur doit être informé lorsqu'un nouveau tweet est analysé.

Ce qui nous intéresse concrètement est de pouvoir lier une localisation donnée à un email. Une fois encore le percolateur peut être utilisé à cette fin.

Nous n'allons envoyer que des latitudes et des longitudes dans les documents que nous allons percoler.

Le Mapping sera donc celui-ci :

```
curl -XPUT 'localhost:9200/subscription' -d '{
  "mappings":{
    "subscriptions":{
      "properties":{
        "geo": {
          "type": "geo_point"
        }
      }
    }
  }
}
```

Pour savoir si un nouveau tweet doit être notifié à une ou plusieurs personnes, nous allons, à chaque nouvel abonnement, stocker dans un nouveau percolateur une requête de recherche concernant une distance géographique de 5km autour d'une latitude et d'une longitude donnée:

```
curl -XPUT localhost:9200/subscription/.percolator/tata@yopmail.com -d '{
  "query" : {
    "filtered" : {
      "filter" : {
        "geo_distance" : {
          "distance" : "5km",
          "geo" : {
            "lat" : "48.8514093" ,
            "lon" : "2.3712249"
          }
        }
      }
    }
  }
}
```

A chaque fois qu'un document sera percolé si il contient un point géographique inclus dans une ou plusieurs des requêtes stockée, l'ensemble des index de ces percolateurs sera renvoyée, permettant ainsi de notifier (envoyer un mail) aux personnes concernée.

Le script de récupération des données :

Deux routines ont donc été mises en places pour la récupération des données, l'une pour les tweets français, l'autre pour les tweets anglais :

```
*/20 * * * * php /var/www/crawl.php fr
```

```
10-59/20 * * * * php /var/www/crawl.php en
```

Elles sont lancées toutes les vingt minutes avec un décalage de 10minutes entre elles à cause des limitations de l'API

Les données sont récupérées sur twitter via leur API. Certaines données géographiques seront traitées par l'api mapquest Deux comptes ont donc été créés afin de pouvoir les requêter.

```
<?php
```

```
require_once('vendor/j7mbo/twitter-api-php/TwitterAPIExchange.php');
```

```
$openmapquestapiKey = "Fmjtd%7Cluu8210b20%2Cr2%3Do5-94r2u0";
```

```
$mapquestUrl = 'http://open.mapquestapi.com/geocoding/v1/';
```

```
$settings = array(
    'oauth_access_token' => "551109921-
t0hx2Vd19vcKRUHGLQdRJjIeIZzk8vhbwmrQT6fB",
    'oauth_access_token_secret' =>
"wMyQzQK3mZ5u0YRcTyYlFjxCS5WOwchD2lM2IN1gdNNbV",
    'consumer_key' => "jh5YKBPPQECw9UEw48y7c931I",
    'consumer_secret' =>
"zV8b49Cygd9vN6YgkkaniYM0OXjNJC0h5PkbU2N2KFNazjSWhz"
);
```

En fonction de la langue en paramètre la requête effectuée au sein de l'API twitter sera différente :

```
$fr="#Heureux+OR+#heureuse+OR+#joie+OR+#bonheur+OR+#youpi";
$en="#happy+OR+#cheerful+OR+#delighted+OR+#ecstatic+OR+#joyful+OR+#pleased+
OR+#glad";
```

```
if ($argv[1] == "fr"){
    $searchrequest=$fr;
}else if ($argv[1] == "en"){
    $searchrequest=$en;
} else {
    die;
}
```

```
$url = 'https://api.twitter.com/1.1/search/tweets.json';
$getfield = '?q='.$searchrequest.'&count=100';
$requestMethod = 'GET';
```

```
$twitter = new TwitterAPIExchange($settings);
$response=$twitter->setGetfield($getfield)
->buildOauth($url, $requestMethod)
->performRequest();
$twitterReturn = json_decode($response);
//print_r($twitterReturn);
```

Les utilisateurs de twitter n'acceptent pas tous la géolocalisation, aussi il sera nécessaire de définir une fonction permettant d'obtenir une longitude et une latitude à partir de la localisation qu'ils ont indiquée comme étant la leur lors de la création de leur compte ou leur dernière localisation connue.

```
function getLatAndLong($place){  
    global $openmapquestapiKey,$mapquestUrl;  
    $json =  
file_get_contents($mapquestUrl.'address?key='.$openmapquestapiKey.'&location='.urlencode($place));  
    $jsonArr = json_decode($json);  
    if ($jsonArr->results[0]->locations != null) {  
        $a_return["lat"] = $jsonArr->results[0]->locations[0]->lat;  
        $a_return["long"] = $jsonArr->results[0]->locations[0]->lng;  
        $a_return["country"] = $jsonArr->results[0]->locations[0]->adminAreal;  
    } else {  
        $a_return = null;  
    }  
    //var_dump($a_return);  
    return $a_return;  
}
```

Une seconde fonction a été définie afin de rechercher si le tweet contient un élément de la blacklist définie au sein d'elasticsearch :

```
function isBlacklisted($tweet){  
    [...]  
    $index = "blacklist";  
    $request = "curl -XGET localhost:9200/blacklist/domains/_percolate  
-d '{"doc" : { "domain" : "$tweet" } }'";  
    $ci = curl_init();  
    curl_setopt($ci, CURLOPT_URL,  
"http://".$search_host."/".$index."/_percolate/");  
    curl_setopt($ci, CURLOPT_PORT, $port);  
    curl_setopt($ci, CURLOPT_TIMEOUT, 200);  
    curl_setopt($ci, CURLOPT_RETURNTRANSFER, 1);  
    curl_setopt($ci, CURLOPT_FORBID_REUSE, 0);  
    curl_setopt($ci, CURLOPT_CUSTOMREQUEST, 'GET');  
    curl_setopt($ci, CURLOPT_POSTFIELDS, makeQuery($loctostore));  
    $response = curl_exec($ci);  
    $blAnswer = json_decode($response)  
    if ($blAnswer->total != 0) {  
        return true;  
    } else  
        return false;  
}
```

Et enfin une fonction nous permet de déterminer si nous avons un ou des utilisateurs abonnés aux tweets d'une localisation donnée, et si tel est le cas, leur envoyer un email de notification:

```
function hasSuscribed($lat,$lon,$text){
```



```

    $request="'{"doc" : { "geo" : { "$lat" : "'$lat'", "$lon" :
"'. $lon.'" }}}'";
    [...]
    $request = "curl -XGET localhost:9200/blacklist/domains/ percolate
-d '{"doc" : { "domain" : "$tweet" } }'";
    $ci = curl_init();
    curl_setopt($ci, CURLOPT_URL,
"http://".$search_host."/".$index."/subscriptions/_percolate/");
    curl_setopt($ci, CURLOPT_PORT, $port);
    curl_setopt($ci, CURLOPT_TIMEOUT, 200);
    curl_setopt($ci, CURLOPT_RETURNTRANSFER, 1);
    curl_setopt($ci, CURLOPT_FORBID_REUSE, 0);
    curl_setopt($ci, CURLOPT_CUSTOMREQUEST, 'GET');
    curl_setopt($ci, CURLOPT_POSTFIELDS, $request);
    $response = curl_exec($ci);
    $subAnswer = json_decode($response)
    if ($subAnswer->total != 0) {
        foreach ($subAnswer->matches as $match) {
            sendmail($match->_id, $text);
        }
    }
}

```

Les données sont stockées au sein de mongoDB, il faut donc initialiser le connecteur :

```

$connector = new MongoClient();
$db= $connector->twitter;
$collection = $db->tweets;

```

Nous traitons donc les éléments renvoyés par tweeter si ceux-ci ne sont pas blacklistés

```

foreach ( $twitterReturn->statuses as $tweet) {

    if (isBlacklisted($tweet->text) == false ) {

        $tweettostore["id"]= $tweet->id;
        $tweettostore["text"]=$tweet->text;
        $tweettostore["created_at"]=$tweet->created_at;
    }
}

```

Une longue série de traitements conditionnels tentent de récupérer des informations de localisation

```

    if (($tweet->user->location != null) || ($tweet->user-
>profile_location !=null) || ($tweet->user->geo != null) || ( $tweet-
>coordinates != null) || ($tweet->place != null) ) {
        if ($tweet->user->location != null) {
            $tweettostore["location"]=$tweet->user-
>location;

            $GPS=getLatAndLong($tweet->user->location);
            $tweettostore["geo"]["lat"]=$GPS["lat"];
            $tweettostore["geo"]["lon"]=$GPS["long"];
            $tweettostore["country"]=$GPS["country"];
        }
        if ($tweet->user->geo != null) {

```

```

                                $tweettostore["geo"]["lat"]=$tweet->user-
>geo->coordinates[1];
                                $tweettostore["geo"]["lon"]=$tweet->user-
>geo->coordinates[0];
                                }
                                if (($tweet->coordinates != null) && ($tweet->user-
>geo == null)) {
                                $tweettostore["geo"]["lat"]=$tweet-
>coordinates->coordinates[1];
                                $tweettostore["geo"]["lon"]=$tweet-
>coordinates->coordinates[0];
                                }
                                if ($tweet->place != null) {
                                $tweettostore["location"]=$tweet->place-
>name;
                                $tweettostore["country"]=$tweet->place-
>country_code;
                                }
                                } else {
                                $tweettostore["location"]="";
                                $tweettostore["country"]="";
                                }
}

```

Si nous disposons d'informations de localisation, nous enregistrons le tweet en base seulement si celui-ci n'a pas déjà été enregistré.

```

                                if ($tweettostore["geo"]["lat"] != null) {
                                $isIdinserted= array('id' => $tweettostore["id"]);
                                $return = $collection->findOne($isIdinserted);
                                if ($return ==null) {
                                $collection->insert($tweettostore);

```

Et nous notifions les abonnés.

```

                                $hasSuscribed($tweettostore["geo"]["lat"],
                                $tweettostore["geo"]["lon"], $tweettostore["text"]);
                                }
                                unset($tweettostore);
                                }
                                }
                                ?>

```

Le webservice d'envoi des données agrégées :

Le Webservice est appelé pour renvoyer un json des tweets agrégés pour une date donnée :

```

<?php
require('vendor/geohash.php');
$search_domain = "elastic.search.happy.inmyworlds.com";
$port = "9200";
$index = "twitter";
function makeSearchRequest($s_date) {
    $startDate = strtotime('%a %b %d %H:%M:%S %z
%Y',strtotime($s_date));
    $endDate = strtotime('%a %b %d %H:%M:%S %z %Y',strtotime($s_date."
23:59:59"));
    $json_request = '{
        "query":{
            "range":{

```

```

        "created_at":{
            "from":"'".$startDate.'"',
            "to":"'".$endDate.'"
        }
    },
    "aggs":{
        "geo1":{
            "geohash_grid":{
                "field":"geo",
                "precision":7
            },
            "aggs":{
                "tags":{
                    "terms":{
                        "field":"text",
                        "size":10
                    }
                }
            }
        }
    }
}';
return $json_request;
}
$request="";
if (isset($_GET["date"])){
    $date_search= $_GET["date"];
    if ($date_search == "today"){
        $request = makeSearchRequest(date("Y-m-d"));
    } else {
        $request = makeSearchRequest($date_search);
    }
}

$ci = curl_init();
curl_setopt($ci, CURLOPT_URL,
"http://".$search_host."/".$index."/_search");
curl_setopt($ci, CURLOPT_PORT, $port);
curl_setopt($ci, CURLOPT_TIMEOUT, 200);
curl_setopt($ci, CURLOPT_RETURNTRANSFER, 1);
curl_setopt($ci, CURLOPT_FORBID_REUSE, 0);
curl_setopt($ci, CURLOPT_CUSTOMREQUEST, 'GET');
curl_setopt($ci, CURLOPT_POSTFIELDS, $request);
$response = curl_exec($ci);
$arrayToSend = array();
$searchAnswer = json_decode($response);
$geohash=new Geohash;
foreach ($searchAnswer->aggregations->geo1->buckets as $bucket){
    $bucketCount = array();
    foreach ($bucket->tags as $count) {
        foreach ($count as $object){

            $bucketCount[]=array("text" => $object->key,
"count" => $object->doc_count);
        }
    }
    $computed_coords=$geohash->decode("$bucket->key");
    $arrayToSend["places"][]=array("lat"=>$computed_coords[0],
"lon"=>$computed_coords[1], "aggs" =>$bucketCount );
}
header("Content-Type: application/json", true);

```

```

    echo json_encode($arrayToSend);
}

```

Une fonction makeSearchRequest Permet de générer un json formé pour aller demander une double agrégation à elasticsearch avec la date du jour :

```

{
    "query":{
        "range":{
            "created_at":{
                "from":"'".$startDate.'"',
                "to":"'".$endDate.'"
            }
        },
        "aggs":{
            "geo1":{
                "geohash_grid":{
                    "field":"geo",
                    "precision":7
                },
                "aggs":{
                    "tags":{
                        "terms":{
                            "field":"text",
                            "size":10
                        }
                    }
                }
            }
        }
    }
}

```

La recherche est effectuée sur une plage de date comprenant une journée entière.

Cette plage de date est ensuite doublement agrégée, la première sur la géolocalisation avec une précision de 152m x 152m, la seconde devant calculer et renvoyer les 10 termes les plus présents dans l'ensemble des tweets ayant été localisés au sein de cette zone.

Le webservice d'abonnement :

L'abonnement est basé sur la percolation, à chaque fois qu'un utilisateur souhaite s'abonner une requête correspondant à sa géolocalisation et avec son email comme index sera enregistré au sein du percolateur :

```

<?php
[...]
$openmapquestapiKey ="Fmjtd%7Cluu8210b20%2Cr2%3Do5-94r2u0";
$mapquestUrl='http://open.mapquestapi.com/geocoding/v1/';

$town=$_GET["town"];
$email=$_GET["email"];
$jsonGeo =
file_get_contents ($mapquestUrl.'address?key='.$openmapquestapiKey.'&location=' .urlencode ($town) );

```

```

$jsonGeoArr = json_decode($jsonGeo);

function makeQuery($loc){
$espercolator= '{
"query" : {
    "filtered" : {
        "filter" : {
            "geo_distance" : {
                "distance" : "5km",
                "geo" : {
                    "lat" : ' . $loc["geo"]["lat"] . ' ,
                    "lon" : ' . $loc["geo"]["long"] . '
                }
            }
        }
    }
}
}';

return $espercolator;
}

if ($jsonGeoArr->results[0]->locations != null) {
    $loctostore["geo"]["lat"] = $jsonGeoArr->results[0]->locations[0]-
>latLng->lat;
    $loctostore["geo"]["long"] = $jsonGeoArr->results[0]->locations[0]-
>latLng->lng;

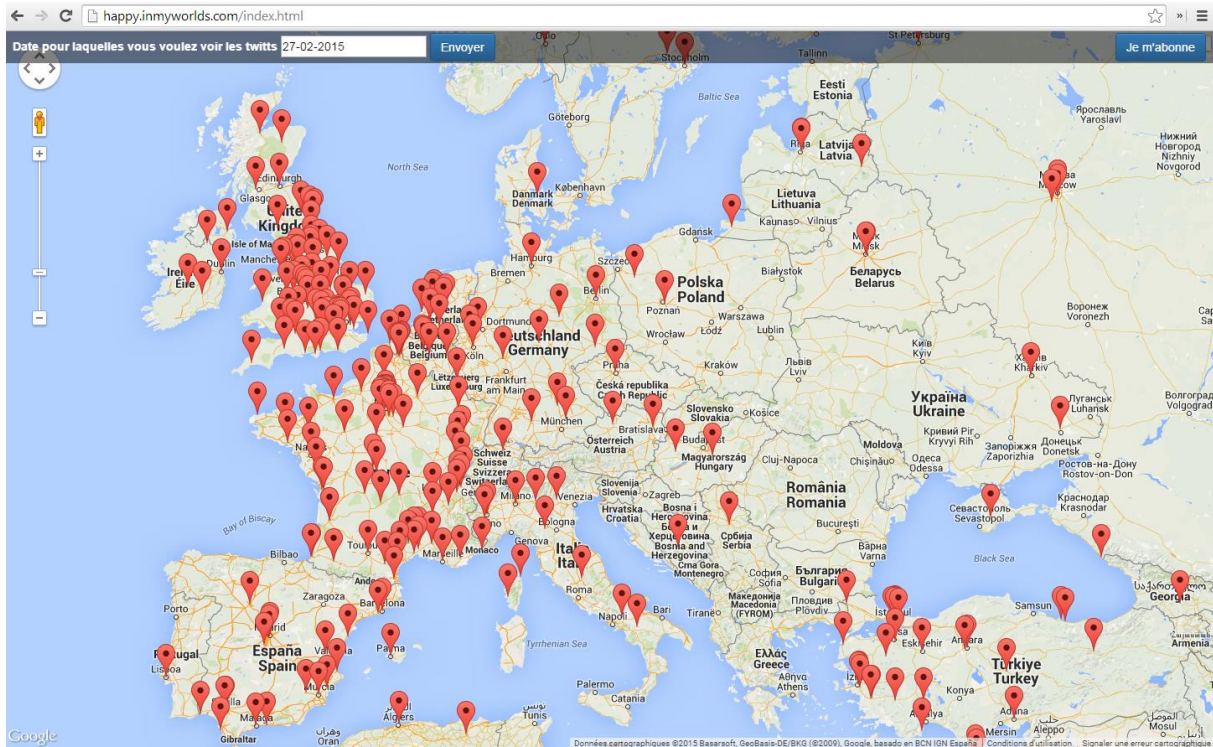
    $ci = curl_init();
    curl_setopt($ci, CURLOPT_URL,
"http://". $search_host . "/" . $index . "/.percolator/" . $email);
    curl_setopt($ci, CURLOPT_PORT, $port);
    curl_setopt($ci, CURLOPT_TIMEOUT, 200);
    curl_setopt($ci, CURLOPT_RETURNTRANSFER, 1);
    curl_setopt($ci, CURLOPT_FORBID_REUSE, 0);
    curl_setopt($ci, CURLOPT_CUSTOMREQUEST, 'PUT');
    curl_setopt($ci, CURLOPT_POSTFIELDS, makeQuery($loctostore));
    $response = curl_exec($ci);
    $result = array('result' => TRUE);
} else {
    $result = array('result' => FALSE);
}

header("Content-Type: application/json", true);
echo json_encode($result);

```

La page Web :

La page web est constituée d'une carte des tweets et d'un header contenant deux formulaires, le premier permettant de filtrer les tweets sur la date, le second permettant de s'abonner aux tweets heureux de votre lieu :



Le webservice est appelé ainsi :

```
function getPlaces() {
    var locations = [];
    var color = ['#A8E58E', '#E29E9E', '#F4E38D', '#82C6E5', '#EA9F9F'];
    var date = $('#dp1').val();
    if(date == '') {
        objDate = new Date();
        month = objDate.getMonth() + 1;
        if(month < 10) {
            month = '0' + month;
        }
        date = objDate.getFullYear() + '-' + (month) + '-' +
objDate.getDate();
    }
    else {
        date = date.split('-');
        date = date[2] + '-' + date[1] + '-' + date[0];
    }
    jQuery.ajax({
        url : 'http://happy.inmyworlds.com/controller.php?date=' + date,
        dataType : 'json',
        success : function(response) {
            // if (response.status == 'OK') {
                places = response.places;
                for (p in places) {
                    place = places[p];
                    var simpleplace = [];
```

```

        simpleplace.push(place.lat);
        simpleplace.push(place.lon);
        var tweetcount="";
        for ( bucket in place.aggs){
            x = Math.floor((Math.random() * 5));
            tweetcount += '<span style="color:' + color[x] +
';font-weight:bold">' + place.aggs[bucket].text + " : " +
place.aggs[bucket].count + "</span><br>";
        }
        simpleplace.push(tweetcount);
        locations.push(simpleplace);
    }
    setMarqueur(locations);
    //}
}
})
return locations;
}

```

Et ils sont chargés sur la map ainsi :

```

function setMarqueur(locations){
    var map = new google.maps.Map(document.getElementById('map'), {
        zoom: 3,
        center: new google.maps.LatLng(48.8514093, 2.3712249),
        mapTypeId: google.maps.MapTypeId.ROADMAP
    });

    var infowindow = new google.maps.InfoWindow();

    var marker, i;
    for (i = 0; i < locations.length; i++) {
        marker = new google.maps.Marker({
            position: new google.maps.LatLng(locations[i][0], locations[i][1]),
            map: map
        });

        google.maps.event.addListener(marker, 'click', (function(marker, i) {
            return function() {
                infowindow.setContent(locations[i][2]);
                infowindow.open(map, marker);
            }
        })(marker, i));
    }
}

```

Ainsi sont affichés l'intégralité des tweets sur une carte du monde.

Conclusion

Travailler avec beaucoup de données peu qualifiées change le point de vue et les façons habituelles de traiter « la data ». Ainsi, sur twitter de très nombreuses données ne sont d'aucune utilité : tagguées au nom de ce qui nous intéresse pour des raisons mercantiles, de nombreux mots communs entre plusieurs langues ont des sens complètement différents.

Et la masse incroyable de données récupérées ou récupérables ne permet plus à l'analyse de s'effectuer directement dans le code.

Mon principal problème au sein de ce développement a été de donner du sens aux mots. C'est tout l'intérêt de la percolation, avec cet outil il serait tout à fait possible de percoler un document contre une requête permettant de déterminer la langue du tweet et ensuite décider de ce qu'il reste à faire. Mais ce module reste à développer.